

# Sisteme de Operare. Laborator 7

Honorius Gâlmeanu  
galmeanu@unitbv.ro

April 11, 2023

## 1 Thread-uri

Thread-urile sunt fire de execuție care rulează concurrent în același proces. De regulă ele partajează variabile și zone de memorie din cadrul procesului, ceea ce face ca scrierea și citirea lor concurrentă din mai multe thread-uri să ducă la suprascrieri ale datelor.

Asemenea proceselor, pe parcursul execuției, thread-urile trec, succesiv, prin stările: **running**, **waiting**, **ready** sau **terminated**. Un thread **running** are controlul CPU și se execută, este activ. **Waiting** este starea în care un thread așteaptă să fie deblocat de către un altul, de obicei prin eliberarea unui mutex sau semafor. Starea **ready** apare atunci când un thread urmează să fie planificat pentru execuție de îndată ce îi va veni rândul. Un thread se află în starea **terminated** dacă și-a încheiat execuția dar părintele său nu i-a inspectat exit code-ul returnat (adică nu a executat un apel `pthread_join` asupra lui).

Deoarece thread-urile au atașate mai puține resurse decât procesele, folosirea lor implică economie de timp CPU (overhead-ul comutării între thread-uri este mult mai mic decât cel asociat comutării între procese). Procesele necesită resurse individuale; thread-urile partajează aceste resurse ale procesului, necesitând astfel mai puțină memorie. Programarea multithreading permite scrierea de aplicații concurente care pot rula atât pe sisteme uniprocessor, cât și multiprocessor, în mod transparent. De asemenea, thread-urile pot îmbunătăți performanța unei aplicații, atunci când aceasta execută frecvent operații ce pot bloca temporar execuția, cum ar fi accesele la resursele I/O.

### 1.1 Funcția `pthread_create`

```
#include <pthread.h>

int pthread_create(pthread_t *thread,
                  const pthread_attr_t *attr,
                  void *(*start_routine) (void *),
                  void *arg);
```

Funcția pornește un thread în procesul apelant, care se va executa concomitent cu thread-ul curent. Thread-ul va funcționa folosind șablonul funcției `start_routine`, căreia îi va trimite argumentul `arg`. Pentru compilare și link-editare pentru toate funcțiile din biblioteca `pthread` se va folosi opțiunea de compilator `-pthread`.

Argumentul `attr` indică atributele cu care se va crea noul thread. O descriere a acestora este disponibilă în pagina de manual `pthread_attr_init`. De regulă acesta se initializează cu `NULL`, caz în care se aplică atributele implicite. În caz de succes, identificatorul thread-ului nou creat este memorat în locația indicată de pointer-ul `thread`, iar rezultatul întors este 0. Dacă funcția a eșuat, un cod de eroare diferit de zero este returnat.

### 1.2 Funcția `pthread_exit`

```
#include <pthread.h>

noreturn void pthread_exit(void *retval);
```

Funcția termină thread-ul care o cheamă și întoarce o valoare folosind variabila `retval`, care poate fi inspectată de funcția `pthread_join` chemată din thread-ul părinte. Terminarea unui thread se poate face și cu instrucțiunea `return`.

### 1.3 Funcția `pthread_join`

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **retval);
```

Funcția așteaptă terminarea thread-ului specificat de variabila `thread` și depune codul de returnare în variabila indicată prin pointer-ul `retval`, dacă acesta nu este `NULL`. Dacă thread-ul este oprit printr-un apel `pthread_cancel`, valoarea `PTHREAD_CANCELED` este pusă în această locație.

## 2 Variabile mutex

### 2.1 Funcțiile `pthread_mutex_init` și `pthread_mutex_destroy`

```
#include <pthread.h>
```

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_mutex_init (pthread_mutex_t *mutex,
                        const pthread_mutexattr_t *attr);
```

Funcția `pthread_mutex_destroy` distruge obiectul mutex referit de variabila `mutex`; structura de date devine astfel neinițializată. Funcția `pthread_mutex_init` va inițializa structura de date referită de pointer-ul `mutex`, folosind atributele definite de `attr`. Dacă aceasta are valoarea `NULL`, se folosesc atributele implicite.

### 2.2 Blocarea și deblocarea unui mutex

```
#include <pthread.h>
```

```
int pthread_mutex_lock (pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock (pthread_mutex_t *mutex);
```

Funcția `pthread_mutex_lock` încearcă obținerea unui mutex și se va bloca dacă mutex-ul este deja obținut de un alt thread, sau întoarce 0 dacă obținerea mutex-ului reușește. Funcția `pthread_mutex_trylock` lucrează ca și cea anterioară doar că va întoarce imediat dacă mutex-ul este deja blocat, returnând un cod de eroare. Funcția `pthread_mutex_unlock` va elibera (release) un mutex care a fost obținut de thread-ul curent printr-un apel `pthread_mutex_lock` anterior.

## 3 Variabile de tip condiție

### 3.1 Crearea și distrugerea

```
#include <pthread.h>
```

```
int pthread_cond_destroy(pthread_cond_t *cond);
int pthread_cond_init (pthread_cond_t *cond,
                      const pthread_condattr_t *attr);
```

Funcțiile creează, respectiv de-inițializează structura folosită pentru variabila de condiție `cond`, care trebuie declarată în prealabil. Dacă se folosește `NULL` pentru variabila pointer `attr`, atunci se folosesc atributele implicite.

## 3.2 Funcțiile pthread\_cond\_wait

```
#include <pthread.h>

int pthread_cond_timedwait(pthread_cond_t *cond,
                           pthread_mutex_t *mutex,
                           const struct timespec *abstime);
int pthread_cond_wait      (pthread_cond_t *cond,
                           pthread_mutex_t *mutex);
```

Funcțiile se folosesc pentru inspectarea variabilei de condiție; dacă aceasta a devenit adevărată, funcția va întoarce, altfel va bloca apelul. Funcțiile eliberează atomic variabila mutex, adică dacă condiția nu se verifică, blocarea se face pe variabila mutex. Varianta cu abstime va întoarce după timpul specificat chiar dacă condiția nu se verifică.

## 3.3 Funcția pthread\_cond\_signal

```
#include <pthread.h>

int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal   (pthread_cond_t *cond);
```

Funcția pthread\_cond\_broadcast va debloca toate thread-urile blocate pe variabila de condiție cond. Varianta sa pthread\_cond\_signal va debloca cel puțin un thread blocat pe acea variabilă de condiție.

# 4 Semafoare

## 4.1 Funcțiile sem\_init și sem\_destroy

```
#include <semaphore.h>

int sem_init  (sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);
```

Realizează inițializarea și dealocarea structurii semafor, care trebuie declarată în prealabil. Variabila pshared specifică modul de partajare cu alte procese, și de regulă se inițializează cu 0 dacă nu se partajează cu alte procese. Variabila value indică valoarea inițială a semaforului.

## 4.2 Funcțiile sem\_wait

```
#include <semaphore.h>

int sem_wait      (sem_t *sem);
int sem_trywait   (sem_t *sem);
int sem_timedwait(sem_t *restrict sem,
                  const struct timespec *abs_timeout);
```

Se folosesc pentru decrementarea semaforului; pot bloca dacă valoarea semaforului deja a ajuns la zero când se realizează apelul. Dacă valoarea semaforului nu este zero, funcția face decrementarea acestuia. Funcția sem\_trywait nu va bloca chiar dacă valoarea semaforului este deja zero, ci va întoarce un cod de eroare diferit de zero. Varianta sem\_timedwait va returna după perioada de timp specificată, dacă call-ul obișnuit ar fi blocat apelul.

## 4.3 Funcția sem\_post

```
#include <semaphore.h>

int sem_post(sem_t *sem);
```

Se incrementează semaforul. Dacă există thread-uri blocate pe acel semafor, unul din ele este trezit și se execută întoarcerea din `sem_wait` (sau variantele sale) în acel thread.

## 4.4 Funcția `sem_getvalue`

```
#include <semaphore.h>
```

```
int sem_getvalue(sem_t *sem, int *sval);
```

Funcția întoarce valoarea curentă a semaforului `sem` în variabila pointer `sval`. Dacă există thread-uri sau procese care așteaptă pe acel semafor, valoarea întoarsă va fi 0. Linux nu întoarce numere negative (ce pot reprezenta numărul de thread-uri care așteaptă).

## 5 Exemple

Se compilează exemplele cu:

Command Line

```
make
```

### 5.1 Programul `hello`

Programul exemplifică utilizarea funcțiilor de creare a thread-urilor. El se execută astfel:

```
$ ./hello
```

### 5.2 Programul `prodcons`

Programul exemplifică utilizarea mutex-urilor și a thread-urilor pe problema clasică producător-consumator. El se execută cu:

```
$ ./prodcons
```

### 5.3 Programul `prodcons`

Programul exemplifică utilizarea mutex-urilor și a semafoarelor pe problema clasică producător-consumator. Se exemplifică cum se evită busy waiting-ul. Programul se execută cu:

```
$ ./semprodcons
```

## 6 Exerciții

### Exercițiul 1

Rulați toate programele prezentate. Asigurați-vă că le-ați înțeles funcționarea. Folosiți-vă de pagina 3 de manual (comanda `man`). În caz că nu găsiți paginile de manual, puteți să le căutați și la <https://man7.org/linux/man-pages/index.html>

#### Exercițiul 2

Rulați programul `hello` de mai multe ori. Ce nedeterminare prezintă? Folosiți o variabilă mutex sau un semafor ca să ordonați pornirea celor două thread-uri, corectând problema afișării.

#### Exercițiul 3

Observați încărcarea CPU-ului provocată de execuția programului `prodcons`, folosind comanda `top`. Modificați programul adăugând o variabilă de condiție astfel ca să eliminați busy waiting-ul care provoacă această încărcare.

#### Exercițiul 4

Modificați programul `prodcons` astfel încât să aveți mai mulți producători și un singur consumator.

#### Exercițiul 5

Modificați programul `semprodcons` astfel încât să aveți mai mulți consumatori și un singur producător.